

JSON Loaders Documentation

1 Introduction

The `json_loaders.py` module loads CTCC configuration from a single combined JSON file when `CTCC_INPUT_MODE=json`. It provides a singleton `JSONDataSource` that holds the combined JSON in memory and exposes `get_data(key)` for section access. A set of `load_*` functions mirror `yaml_loaders.py`: each maps a JSON key (e.g. `01_project_technical_details`) to the same dataclasses and structures used in YAML mode, so calculation scripts can use the same API regardless of input mode.

2 Method Overview

2.1 Purpose

1. Provide a singleton `JSONDataSource` that stores the combined JSON; data can be set via `set_json_data()` or *auto-loaded from CTCC_JSON_DATA_FILE when `get_data()` is first called.*
1. Expose `get_data(key)` to return the dict for a given top-level key (e.g. `03_financing`).
2. Provide `load_*` functions that call `get_data()` with the appropriate key and map the dict into the same dataclasses as `yaml_loaders` (e.g. `ProjectTechnicalDetails`, `FinancingDetails`, `CongestionCurtailmentParams`).
3. Use `calculation_utils` and `yaml_loaders` for normalization (e.g. `normalize_capacity_mw`, `normalize_construction_type`) and type definitions; `financial_utils` for WACC when loading financing.

2.2 Calculation Approach

I/O only; no cost/benefit formulas. Keys in the combined JSON match YAML file names (e.g. `01_project_technical_details`, `17_congestion_curtailment_reductions`). There is no scenario ID in the loader; the combined JSON is assumed to represent one scenario. Scenario selection, if needed, is handled by the caller (e.g. server or CLI) before setting the JSON data.

2.3 Inputs and Outputs

Inputs:

- Combined JSON: either set programmatically via `set_json_data(combined_json)` or loaded from the file.
- Top-level keys in the JSON correspond to YAML section names (e.g. `01_project_technical_details`, `02_project_physical_details`, `03_financing`, `17_congestion_curtailment_reductions`, `22_primary_bcr_config`).

Outputs:

- In-memory dicts or dataclass instances returned to callers (calculation scripts); no files written by this module.

2.4 Integration with Other Scripts

- `smart_loaders.py` — When `CTCC_INPUT_MODE=json`, imports all `load_*` and `get_data` from `json_loaders`; calculation scripts then call `load_project_technical_details()`, `load_financing_details()`, etc., without knowing the backend.
- `yaml_loaders.py` — Supplies dataclass and type definitions (`ProjectTechnicalDetails`, `CongestionCurtailmentParams`, `PhysicalDetailsDetailed`, `CircuitAndResistanceDetails`, `FinancingDetails`) and `normalize_construction_type`.
- `calculation_utils.py` — `normalize_capacity_mw`, `get_converter_type`.
- `financial_utils.py` — `calculate_real_wacc` used in `load_financing_details`.

3 Key Functions

3.1 JSONDataSource

Singleton class that holds the combined JSON. `get_data(key)` returns `self._json_data[key]`; if `_json_data` is `None`, it first tries `_load_from_env_file()` (read from `CTCC_JSON_DATA_FILE`).

Key logic (`get_data`):

```
1 if self._json_data is None:
2     self._load_from_env_file()
3 if self._json_data is None:
4     raise RuntimeError("JSON data not set...")
5 if key not in self._json_data:
6     raise KeyError(f"Key '{key}' not found in JSON data.")
7 return self._json_data[key]
```

Key logic (`_load_from_env_file`):

```

1 json_file_path = os.environ.get("CTCC_JSON_DATA_FILE")
2 if json_file_path and os.path.exists(json_file_path):
3     with open(json_file_path, "r") as f:
4         self._json_data = json.load(f)

```

3.2 set_json_data(combined_json)

Sets the combined JSON dict on the global JSONDataSource instance so all load_* functions use it.

3.3 load_project_technical_details()

Loads 01_project_technical_details, normalizes construction type and capacity, and returns ProjectTechnicalDetails (same as yaml_loaders).

Key logic:

```

1 project_details = _data_source.get_data("01
   _project_technical_details")
2 pd = project_details["project"]
3 tl = project_details["timeline"]
4 construction_type = normalize_construction_type(pd["
   construction_type"])
5 capacity_mw = normalize_capacity_mw(pd["capacity_mw"])
6 # ... converter_type, row_agreement_type, etc.
7 return ProjectTechnicalDetails(
8     construction_type=construction_type,
9     ac_dc=pd["ac_dc"],
10    capacity_mw=capacity_mw,
11    ...
12 )

```

3.4 load_financing_details()

Loads 03_financing, computes real WACC via calculate_real_wacc, returns FinancingDetails.

3.5 load_congestion_curtailement_reductions()

Loads 17_congestion_curtailement_reductions; checks 01_project_technical_details for reconductoring to choose greenfield_* vs reconductoring_* section. Returns CongestionCurtailement with flow factor, binding hours, prices, curtailment parameters, etc.

3.6 load_primary_bcr_config()

Loads 22_primary_bcr_config and returns a dict of module enable/disable flags (operational, risk, energy, benefits). If the section is missing or invalid, returns default (all enabled)

and optionally warns.

3.7 Other load_* functions

- `load_physical_details()` — `02_project_physical_details`; returns total miles (sum of `terrain.terrain_miles` values).
- `load_physical_details_detailed()` — returns `PhysicalDetailsDetailed` with per-terrain miles.
- `load_circuit_and_resistance_details(category)` — `21_project_category_circuit_and_resist` returns `CircuitAndResistanceDetails` for the given category.
- `load_row_widths(category)`, `load_row_details()`, `load_delay_costs()`, `load_emissions_details()`, `load_contingencies()`, `load_environmental_mitigation()`, `load_cost_timing_patterns()`, `load_afudc_config()`, `load_insurance_details()`, `load_wildfire_costs()`, `load_outage_costs()`, `load_terrain_data()` — each calls `get_data` with the corresponding key and returns the section dict or derived values.

4 Example

Combined JSON top-level keys (same as YAML file names):

```
1 {  
2   "01_project_technical_details": { "project": {...}, "timeline":  
3     {...} },  
4   "02_project_physical_details": { "terrain": {...} },  
5   "03_financing": { "financial": {...} },  
6   "17_congestion_curtailment_reductions": { "greenfield_...": {...},  
7     "reconductoring_...": {...} },  
8   "22_primary_bcr_config": { "primary_bcr_config": {...} }  
9 }
```

Using the loader (e.g. in a calculation script):

```
1 # Set by caller or auto-load from CTCC_JSON_DATA_FILE  
2 from smart_loaders import load_project_technical_details,  
3   load_financing_details  
4 details = load_project_technical_details()  
5 financing = load_financing_details()
```